

*Please check the notes section for each slide in all slide decks uploaded now, in the past or in the future.
In case of any discrepancies in the slide content, please feel free to reach out.*

Inheritance

Mohammad Salman

Dean's Fellow

Let's Implement Inheritance!

Public Inheritance is what we'll primarily look at in this course.

<https://www.learncpp.com/cpp-tutorial/basic-inheritance-in-c/>

Recap of some basic ideas from the first part...

- *A derived class **extends the capabilities** of its base class.*
- *A derived class is typically **more specialized** than its base class.*
- *You can have **multiple levels** of inheritance (chaining inheritance).*

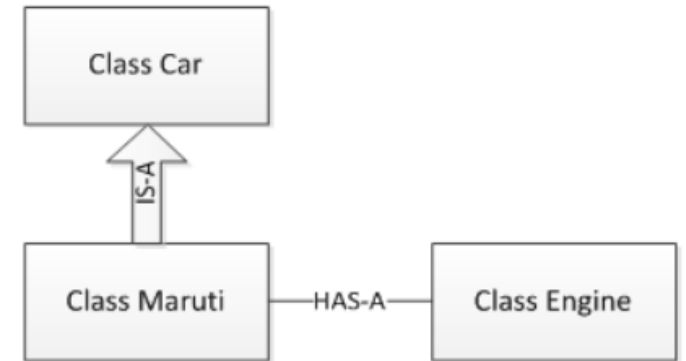
Understanding 'Has-a' vs 'Is-a' Relationships

- "Is-a" Relationship

- Indicates **inheritance** or **generalization**.
- Suggests that one class is a **specialized version** of another.
- Represented using **inheritance** in object-oriented programming.

- "Has-a" Relationship

- Indicates **composition** or **aggregation**.
- Suggests that one class **contains** or is associated with another as a part of its data or functionality.
- Represented using **fields or properties** in object-oriented programming.



Key Differences:

Aspect	Is-a (Inheritance)	Has-a (Composition/Aggregation)
Meaning	A class is a specialized type of another.	A class contains another as a part.
Relationship Type	Generalization or specialization.	Part-of or contains relationship.
Coupling	Higher coupling; changes in the base class affect derived classes.	Lower coupling; changes in a contained class don't affect the container.
Code Reuse	Achieved by inheriting behavior.	Achieved by composing behavior.

Public Inheritance

- There are other types of inheritance as well.
- **Public inheritance** is generally the more common one and easy to understand as well.
 - The access levels of the inherited members within the derived class is the same as the access levels of those members in the base class.

Let's say we have a `class Person`.

Objects of this class would be a generic person. The members of this class reflect that.

```
class Person
{
private:
    std::string m_name;
    int m_age;

public:
    Person()
        : m_name{ "Unnamed" }, m_age{ -1 }
    {}

    Person(const char name[], int age = -1)
        : m_name{ name }, m_age{ age }
    {}

    const std::string& getName() const
    { return m_name; }

    int getAge() const
    { return m_age; }

};
```

```
int main()
{
    Person daffy{ "Daffy Duck", 86 };

    std::cout << daffy.getName() << ", "
               << daffy.getAge();

    return 0;
}
```

Note

You can write this code in your editor, and hover over the string literal in `main()` to see its data type.

If I want to represent a cricketer, I can extend the **Person** class using (public) inheritance to include cricketer specific information.

So now an object of **CricketPlayer** will inherit all members from the **Person** class, and have its own members as well – **Person** has been specialized into a **CricketPlayer**.

```
class Person
{
private:
    std::string m_name;
    int m_age;

public:
    Person()
        : m_name{ "Unnamed" }, m_age{ -1 }
    {}

    Person(const char name[], int age = -1)
        : m_name{ name }, m_age{ age }
    {}

    const std::string& getName() const
    { return m_name; }

    int getAge() const
    { return m_age; }

};
```

```
class CricketPlayer : public Person
{
private:
    double m_strikeRate;
    int m_totalRuns;

public:
    CricketPlayer(double sr = 0.0, int runs = 0)
        : m_strikeRate{sr}, m_totalRuns{runs}
    {}

};
```

*One way to achieve this result could be to just copy-paste the **Person**'s members in **CricketPlayer** – bad idea.*

Between composition and inheritance, inheritance makes conceptual sense here.

*A **CricketPlayer** is-a **Person**.*

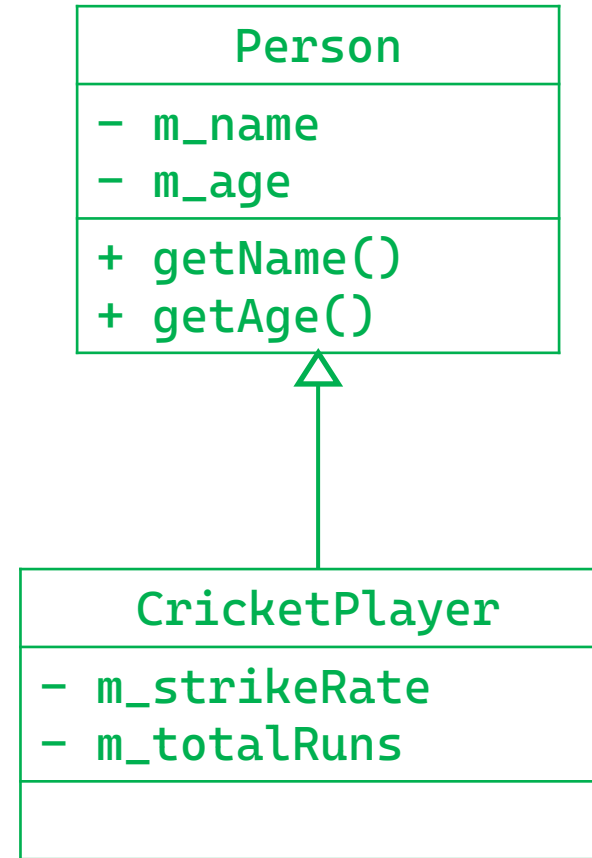
You'll see that this program runs fine.
The inherited member, `getName()`, is being used.

Unnamed is received as an output.

```
int main()
{
    // Create a new CricketPlayer object
    CricketPlayer abcd;

    std::cout << abcd.getName();

    return 0;
}
```



Key

- private members
+ public members

Try to understand this output. Hopefully, by the end of the week, you should be in a good spot to figure this out.

Order of Construction and Destruction of Derived Classes

*Closer look at the **order of construction and destruction** for a **derived class**.*

Please do read the article at the following link.

<https://www.learncpp.com/cpp-tutorial/order-of-construction-of-derived-classes/>

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

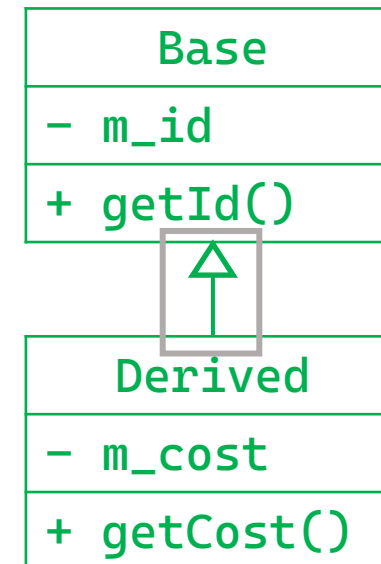
    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

This is not a fully specified UML Class Diagram, but it still reveals a lot of information about,

- the class definitions of **Base** and **Derived**, and
- the relationship between them (which is inheritance in this example).



```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

We consider an object of the **Derived** class as a multi-part object. One part is its own, and the rest belong to each of its parent(s) in its inheritance chain.

In this example, these are its parts:

- one part **Base**.
- one part **Derived**.

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

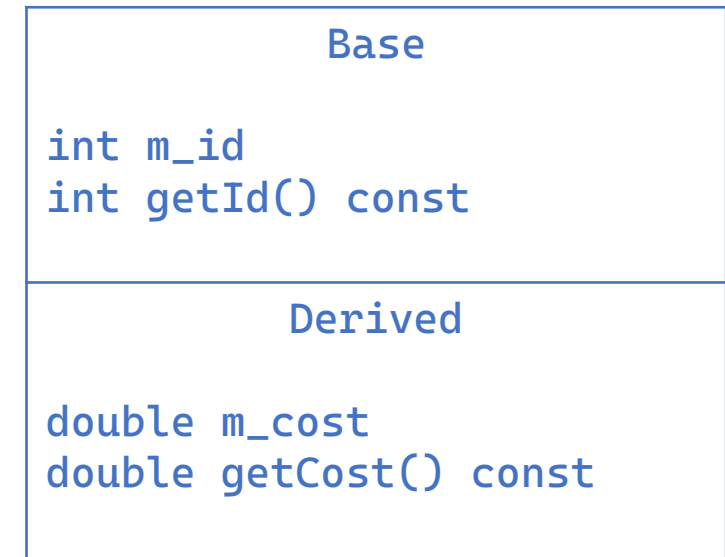
    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

Derived d;



```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

Let's revisit how objects are created.

We will create an object of class **Base** first.

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

Output

```

int main()
{
    Base b1;

    return 0;
}

```

Firstly, `main()` is called.

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

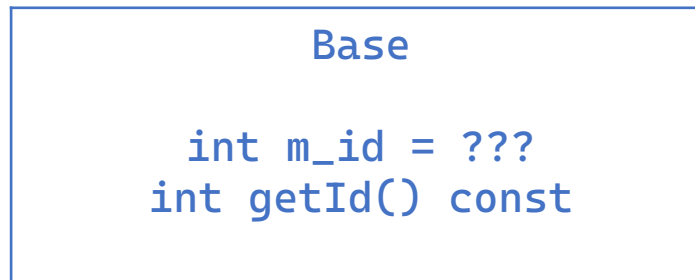
    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output



Memory for the object is
allocated by the compiler.

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

```

int main()
{
    Base b1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

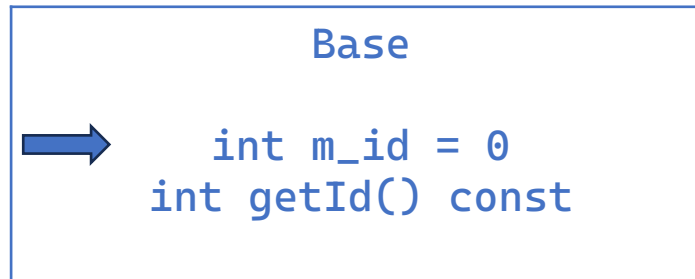
    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

Output



Ctor gets called and
initialization happens.

```

int main()
{
    Base b1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

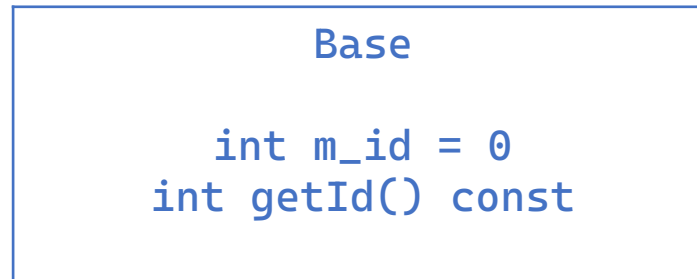
    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output
Default Base



Body of the ctor executes.

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

```

int main()
{
    Base b1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output
Default Base

```

class Derived : public Base
{
private:
    double m_cost;

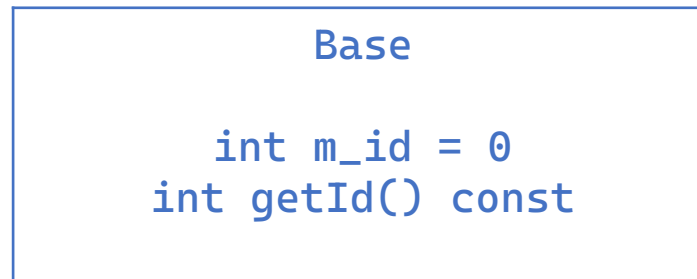
public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```



The control is returned back to
main().

```

int main()
{
    Base b1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

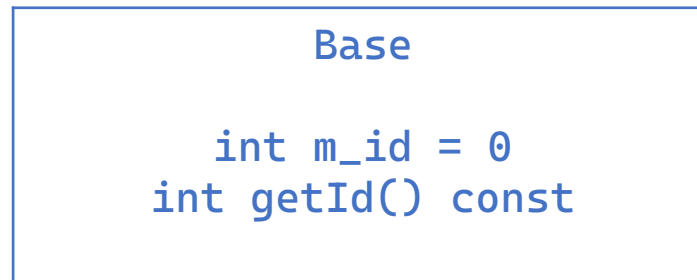
    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output
Default Base
Base destructor



Compiler automatically will attempt to deallocate memory occupied by **b1**.
So its destructor is called.

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

```

int main()
{
    Base b1;

    return 0;
}

```

The Construction Process

1. Memory for **b1** is set aside.
2. The appropriate **Base** constructor is called.
3. The member initializer list initializes variables.
4. The body of the constructor executes.
5. Control is returned to the caller.

```
int main()
{
    // use Base() constructor
    Base b1;

    return 0;
}

Base::Base()
    : m_id { 0 }
{
    std::cout << "Base\n";
}
```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

Output

Now we will see the construction of a **Derived** object.

```

int main()
{
    Derived d1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output

```

class Derived : public Base
{
private:
    double m_cost;

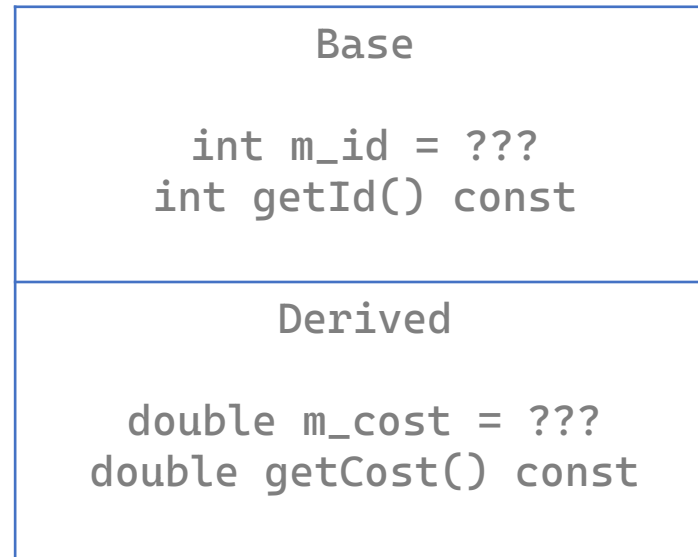
public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```



```

int main()
{
    Derived d1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output

```

class Derived : public Base
{
private:
    double m_cost;

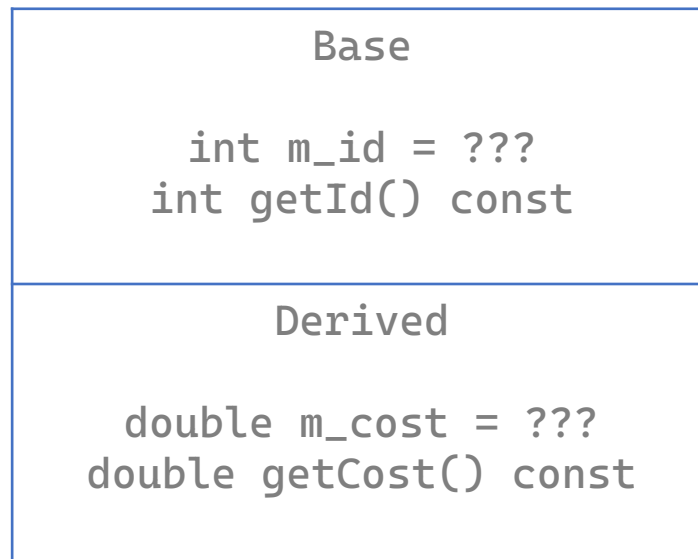
public:
    ➔ Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```



```

int main()
{
    Derived d1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
➔ Base()
    : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output

You may ask why the default version of Base() gets called. This will become clearer when we start the next phase of our discussion. This is because we haven't told the compiler which version to call.

```

class Derived : public Base
{
private:
    double m_cost;

public:
➔ Derived()
    : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

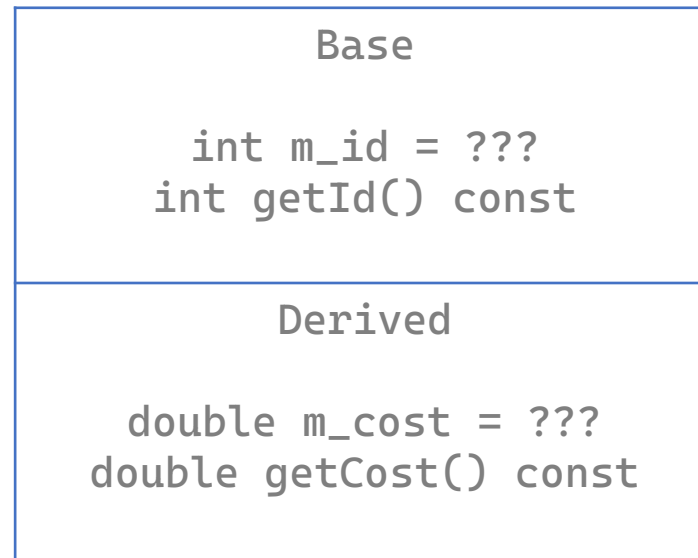
    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

Even the initialization doesn't happen.



```

int main()
{
    Derived d1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    ➔ Base()
        : m_id { 0 }
        { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
        { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output
Default Base

```

class Derived : public Base
{
private:
    double m_cost;

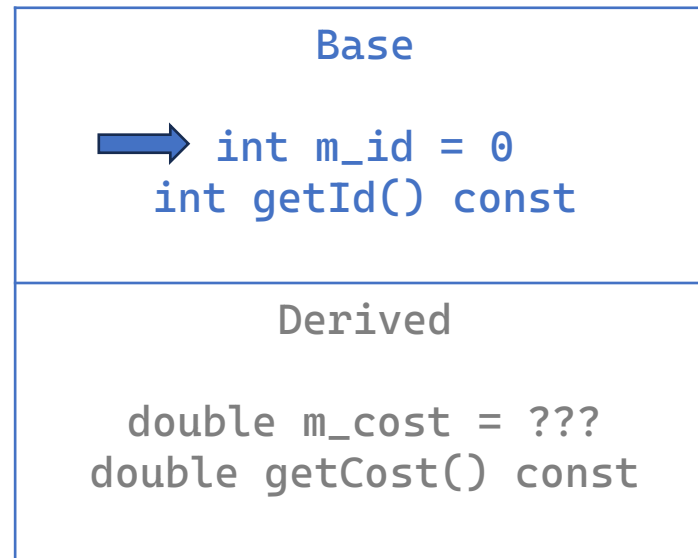
public:
    ➔ Derived()
        : m_cost { 0.0 }
        { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
        { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```



```

int main()
{
    Derived d1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output
Default Base
Default Derived

```

class Derived : public Base
{
private:
    double m_cost;

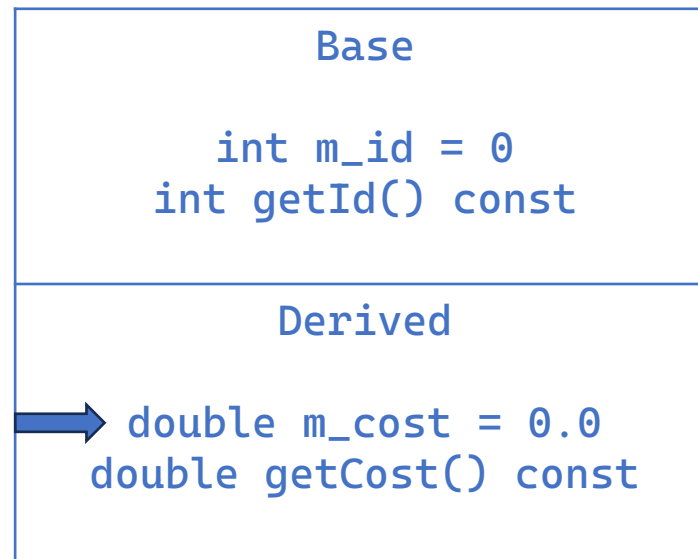
public:
    → Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```



```

int main()
{
    Derived d1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

Output
Default Base
Default Derived

```

class Derived : public Base
{
private:
    double m_cost;

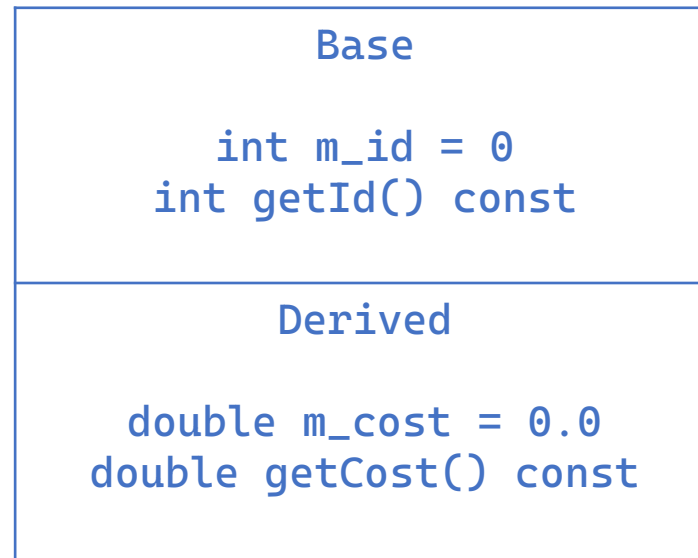
public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```



```

int main()
{
    Derived d1;

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived : public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(double cost)
        : m_cost { cost }
    { std::cout << "One-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

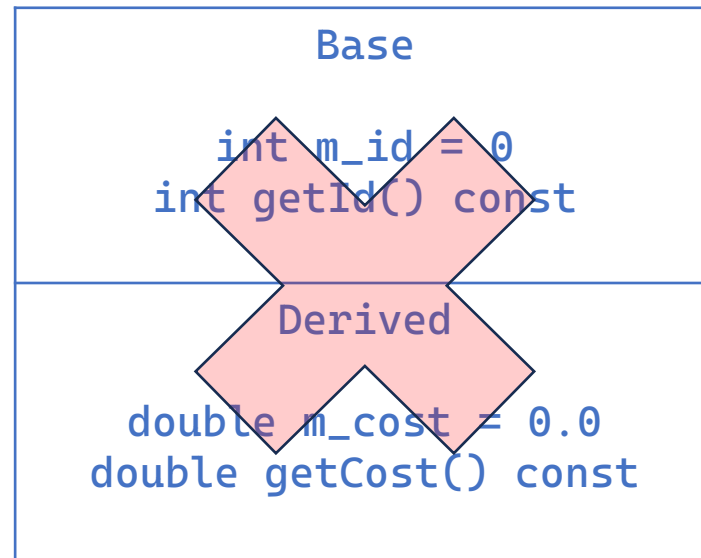
    double getCost() const { return m_cost; }
};

```

Output

Default Base
 Default Derived
 Derived destructor
 Base destructor

Destruction happens in the reverse order.



```

int main()
{
    Derived d1;

    return 0;
}

```

The Construction Process

1. Memory for **d1** is set aside (enough for both the **Base** and **Derived** portions).
2. The appropriate **Derived** constructor is called.
3. The **Base** object is constructed first using the appropriate Base constructor.
 - In this case, it is the default constructor because we haven't specified anything.
4. The member initializer list initializes variables.
5. The body of the constructor executes.
6. Control is returned to the **Derived** constructor, which after finishing, returns the control back to the **main()**.

```
int main()
{
    // use Derived(double) constructor
    Derived d1;

    return 0;
}

Derived::Derived()
    : m_cost { 0.0 }
{
    std::cout << "Derived\n";
}

Base::Base()
    : m_id { 0 }
{
    std::cout << "Base\n";
}
```

Destruction of derived objects

- When an object of a derived class is destroyed, each destructor is called in the **reverse order of construction**.
- *Following will come later.*

Warning

If your base class has virtual functions, your destructor should also be virtual, otherwise undefined behavior will result in certain cases. We cover this case in lesson 25.4 -- Virtual destructors, virtual assignment, and overriding virtualization.

Initializing Derived Classes

Please do read the article at the following link.

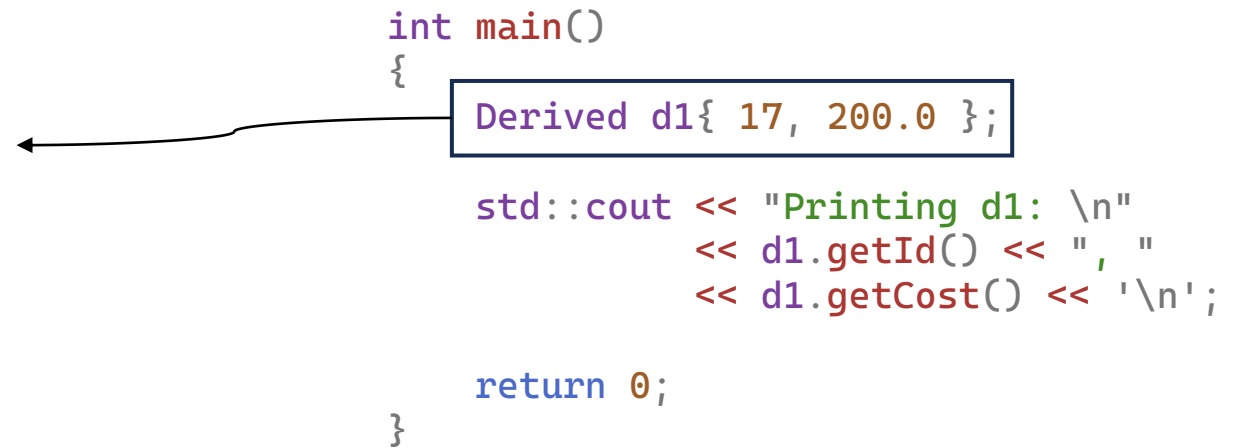
<https://www.learncpp.com/cpp-tutorial/constructors-and-initialization-of-derived-classes/>

Initializing Inherited Members

- We saw that when creating the **Derived**-type object, the compiler implicitly called the default **Base** constructor.
- We can control which variant of the parent constructor gets called.
- This helps with **initializing the inherited members**.

In order to create **d1**, I'm providing two initializing values.

So I would need a **two-arg ctor** in the **Derived** class.



```
int main()
{
    Derived d1{ 17, 200.0 };

    std::cout << "Printing d1: \n"
               << d1.getId() << ", "
               << d1.getCost() << '\n';

    return 0;
}
```

```

class Derived: public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
        { std::cout << "Default Derived\n"; }

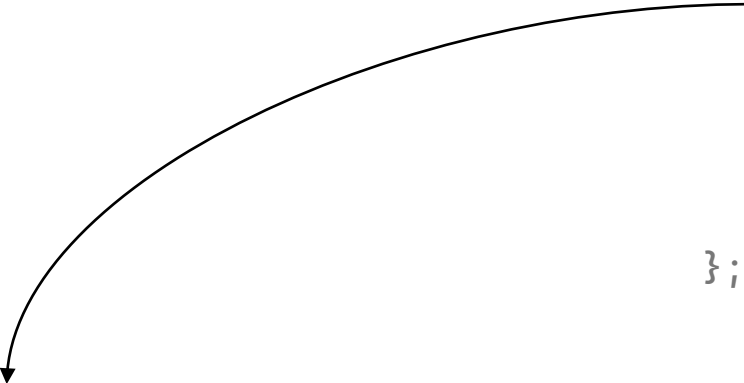
    Derived(int id, double cost)
        : m_cost{ cost }
        { std::cout << "Two-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

I have updated the one-arg ctor so that it can instead accept two arguments.



```

int main()
{
    Derived d1{ 17, 200.0 };

    std::cout << "Printing d1: \n"
              << d1.getId() << ", "
              << d1.getCost() << '\n';

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

The **Base** class stays untouched for this example.

```

class Derived: public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(int id, double cost)
        : m_cost{ cost }
    { std::cout << "Two-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

```

int main()
{
    Derived d1{ 17, 200.0 };

    std::cout << "Printing d1: \n"
              << d1.getId() << ", "
              << d1.getCost() << '\n';

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

When the compiler sees our intent to instantiate **d1**, it will call the appropriate ctor *(after allocating sufficient memory for it)*.

Within the ctor, **id = 17, cost = 200.0**
Note that these variables are local to the ctor.

```

class Derived: public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(int id, double cost)
        : m_cost{ cost }
    { std::cout << "Two-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

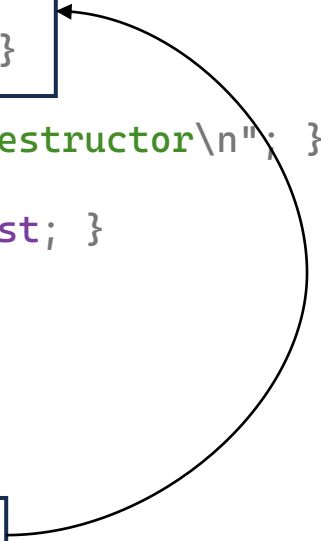
```

int main()
{
    Derived d1{ 17, 200.0 };

    std::cout << "Printing d1: \n"
              << d1.getId() << ", "
              << d1.getCost() << '\n';

    return 0;
}

```



```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived: public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(int id, double cost)
        : m_cost{ cost }
    { std::cout << "Two-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

Before doing anything in **Derived()**, the compiler goes to its parent's ctor.

In this case, it goes to **default** parent ctor because we didn't give the compiler any relevant information on where to go.

So, in this case, **d1.m_id** is now initialized to **0** instead of **17**.
Note that d1.m_cost still hasn't been initialized.

```

int main()
{
    Derived d1{ 17, 200.0 };

    std::cout << "Printing d1: \n"
              << d1.getId() << ", "
              << d1.getCost() << '\n';

    return 0;
}

```

This is how we tell the compiler which ctor to call.

We make the call to the relevant parent ctor within the member initializer list of the child ctor.

```
class Derived: public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(int id, double cost)
        : Base{ id }, m_cost{ cost }
    { std::cout << "Two-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};
```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived: public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(int id, double cost)
        : Base{ id }, m_cost{ cost }
    { std::cout << "Two-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

Making this one change shifts the flow of the program.

Based on your member init list, the compiler now goes to the **one-arg ctor** in the parent class.

```

int main()
{
    Derived d1{ 17, 200.0 };

    std::cout << "Printing d1: \n"
              << d1.getId() << ", "
              << d1.getCost() << '\n';

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived: public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(int id, double cost)
        : Base{ id }, m_cost{ cost }
    { std::cout << "Two-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

And so the cool thing about this is that `id = 17` is now passed to `Base(int)` ctor.

So it can use `id` to initialize the inherited member, `m_id`, of `d1` to `17`, which was supplied as an initializing value.

In other words, `d1.m_id` is now initialized to `17`.

```

int main()
{
    Derived d1{ 17, 200.0 };

    std::cout << "Printing d1: \n"
              << d1.getId() << ", "
              << d1.getCost() << '\n';

    return 0;
}

```

```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived: public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(int id, double cost)
        : Base{ id }, m_cost{ cost }
    { std::cout << "Two-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

After the parent ctor finishes execution, control is returned back to the child ctor.

It then initializes the `m_cost` attribute to `200.0`.

And then the control is returned back to `main()`.

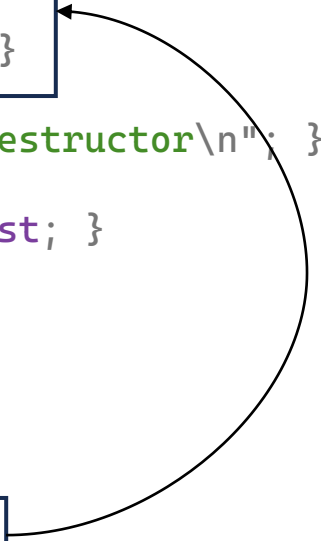
```

int main()
{
    Derived d1{ 17, 200.0 };

    std::cout << "Printing d1: \n"
              << d1.getId() << ", "
              << d1.getCost() << '\n';

    return 0;
}

```



```

class Base
{
private:
    int m_id;

public:
    Base()
        : m_id { 0 }
    { std::cout << "Default Base\n"; }

    Base(int id)
        : m_id { id }
    { std::cout << "One-arg Base\n"; }

    ~Base() { std::cout << "Base destructor\n"; }

    int getId() const { return m_id; }
};

```

```

class Derived: public Base
{
private:
    double m_cost;

public:
    Derived()
        : m_cost { 0.0 }
    { std::cout << "Default Derived\n"; }

    Derived(int id, double cost)
        : Base{ id }, m_cost{ cost }
    { std::cout << "Two-arg Derived\n"; }

    ~Derived() { std::cout << "Derived destructor\n"; }

    double getCost() const { return m_cost; }
};

```

Order of construction { One-arg Base
Two-arg Derived
Printing d1:

Initialization { 17, 200

Order of destruction { Derived destructor
Base destructor

```

int main()
{
    Derived d1{ 17, 200.0 };

    std::cout << "Printing d1: \n"
              << d1.getId() << ", "
              << d1.getCost() << '\n';

    return 0;
}

```

(SS) Why will the following modification to the child ctor not work?

Go read :P

<https://www.learncpp.com/cpp-tutorial/constructors-and-initialization-of-derived-classes/>

One of the current shortcomings of our Derived class as written is that there is no way to initialize m_id when we create a Derived object. What if we want to set both m_cost (from the Derived portion of the object) and m_id (from the Base portion of the object) when we create a Derived object?

New programmers often attempt to solve this problem as follows:

```
1 | class Derived: public Base
2 | {
3 | public:
4 |     double m_cost {};
5 |
6 |     Derived(double cost=0.0, int id=0)
7 |         // does not work
8 |         : m_cost{ cost }
9 |         , m_id{ id }
10 |    {
11 |    }
12 |
13 |     double getCost() const { return m_cost; }
14 | };
```

This is a good attempt, and is almost the right idea. We definitely need to add another parameter to our constructor, otherwise C++ will have no way of knowing what value we want to initialize m_id to.

However, C++

(SS) Why will the following modification to the child ctor not work?

Go read (same link as on the previous slide) :P

Also, apart from the reasons mentioned in the article, note that in our example, the parent's members were private, which means they can only be accessed within the member functions of the parent class. So will the following work in that case?

*If you think deeply, you'll intuit the purpose of **protected**, the third access specifier (TBD later).*

```
1  class Derived: public Base
2  {
3  public:
4      double m_cost {};
```



```
6      Derived(double cost=0.0, int id=0)
7          : m_cost{ cost }
8      {
9          m_id = id;
10     }
11
12     double getCost() const { return m_cost; }
13 };
```

(SS) After calling the parent ctor from within the member init list of the child ctor.
Another example with detailed steps. There is yet another example that uses different classes.

```
1 | #include <iostream>
2 |
3 | int main()
4 | {
5 |     Derived derived{ 1.3, 5 }; // use Derived(double, int) constructor
6 |     std::cout << "Id: " << derived.getId() << '\n';
7 |     std::cout << "Cost: " << derived.getCost() << '\n';
8 |
9 |     return 0;
10 | }
```

The base class constructor Base(int) will be used to initialize m_id to 5, and the derived class constructor will be used to initialize m_cost to 1.3!

Thus, the program will print:

```
Id: 5
Cost: 1.3
```

In more detail, here's what happens:

1. Memory for derived is allocated.
2. The Derived(double, int) constructor is called, where cost = 1.3, and id = 5.
3. The compiler looks to see if we've asked for a particular Base class constructor. We have! So it calls Base(int) with id = 5.
4. The base class constructor member initializer list sets m_id to 5.
5. The base class constructor body executes, which does nothing.
6. The base class constructor returns.
7. The derived class constructor member initializer list sets m_cost to 1.3.
8. The derived class constructor body executes, which does nothing.
9. The derived class constructor returns.

Note that this example just shows the idea that **inheritance chains**.

```
1 #include <iostream>
2
3 class A
4 {
5 public:
6     A(int a)
7     {
8         std::cout << "A: " << a << '\n';
9     }
10 };
11
12 class B: public A
13 {
14 public:
15     B(int a, double b)
16     : A{ a }
17     {
18         std::cout << "B: " << b << '\n';
19     }
20 };
21
22 class C: public B
23 {
24 public:
25     C(int a, double b, char c)
26     : B{ a, b }
27     {
28         std::cout << "C: " << c << '\n';
29     }
30 };
31
```

```
32 int main()
33 {
34     C c{ 5, 4.3, 'R' };
35
36     return 0;
37 }
```

C c;

A
B
C

5, 4.3, 'R'

Output

A: 5
B: 4.3
C: R

Note that this example just shows the idea that **inheritance chains**.

```
1 #include <iostream>
2
3 class A
4 {
5 public:
6     A(int a)
7     {
8         std::cout << "A: " << a << '\n';
9     }
10 };
11
12 class B: public A
13 {
14 public:
15     B(int a, double b)
16     : A{ a }
17     {
18         std::cout << "B: " << b << '\n';
19     }
20 };
21
22 class C: public B
23 {
24 public:
25     C(int a, double b, char c)
26     : B{ a, b }
27     {
28         std::cout << "C: " << c << '\n';
29     }
30 };
31
```

```
int main()
```

```
{
```

```
    B b{ 5, 4.3 };
```

```
    return 0;
```

```
}
```

5, 4.3

B b;

A

B

Output

A: 5

B: 4.3

What will be the output?

```
#include <iostream> // Example 20.4
using std::cout;

class A
{
public:
    A(int k) {cout << "A:" << k;}
    ~A() {cout << " ~A\n";}
};

class B : public A
{
public:
    B(int m) : A(m+1) {cout << " B:" << m;}
    ~B() {cout << " ~B";}
};

class C : public B
{
public:
    C(int p) : B(2*p) {cout << " C:" << p << '\n';}
    ~C() {cout << " ~C";}
};

int main()
{
    C c(10);
    return 0;
}
```

Output:

A:21 B:20 C:10

Reminder for self-study

- Don't forget to go back to slide 7 of this presentation to understand why the output was **Unnamed**.
- There are some questions in the **Quiz Time** section in the following link for your practice:
 - <https://www.learncpp.com/cpp-tutorial/constructors-and-initialization-of-derived-classes/>

Inheritance and Access Specifiers

We will talk about the third access specifier – protected.

<https://www.learncpp.com/cpp-tutorial/inheritance-and-access-specifiers/>

Question

Can derived classes access the private members of the base class?

The Good Old...

- **private:**
 - *Members are only accessible within the member functions of the same class.*
 - *Or by friends.*
- **public:**
 - *Members are accessible anywhere.*

```
1 class Base
2 {
3     private:
4         int m_private {}; // can only be accessed by Base members and friends (not derived classes)
5     public:
6         int m_public {}; // can be accessed by anybody
7 }
```

New Access Level – **protected**

- **protected:**
 - *Related to inheritance.*
- These members can be accessed within:
 - The member functions of their own class.
 - Friends.
 - Derived classes.
- However, protected members are **not accessible** from outside the class.

```

1 class Base
2 {
3 public:
4     int m_public {}; // can be accessed by anybody
5 protected:
6     int m_protected {}; // can be accessed by Base members, friends, and derived classes
7 private:
8     int m_private {}; // can only be accessed by Base members and friends (but not derived classes)
9 };
10
11 class Derived: public Base
12 {
13 public:
14     Derived()
15     {
16         m_public = 1; // allowed: can access public base members from derived class
17         m_protected = 2; // allowed: can access protected base members from derived class
18         m_private = 3; // not allowed: can not access private base members from derived class
19     }
20 };
21
22 int main()
23 {
24     Base base;
25     base.m_public = 1; // allowed: can access public members from outside class
26     base.m_protected = 2; // not allowed: can not access protected members from outside class
27     base.m_private = 3; // not allowed: can not access private members from outside class
28
29     return 0;
30 }

```

In the above example, you can see that the protected base member `m_protected` is directly accessible by the derived class, but not by the public.

Self Study

So when should I use the protected access specifier?

With a protected attribute in a base class, derived classes can access that member directly. This means that if you later change anything about that protected attribute (the type, what the value means, etc...), you'll probably need to change both the base class AND all of the derived classes.

Therefore, using the protected access specifier is most useful when you (or your team) are going to be the ones deriving from your own classes, and the number of derived classes is reasonable. That way, if you make a change to the implementation of the base class, and updates to the derived classes are necessary as a result, you can make the updates yourself (and have it not take forever, since the number of derived classes is limited).

Making your members private means the public and derived classes can't directly make changes to the base class. This is good for insulating the public or derived classes from implementation changes, and for ensuring invariants are maintained properly. However, it also means your class may need a larger public (or protected) interface to support all of the functions that the public or derived classes need for operation, which has its own cost to build, test, and maintain.

In general, it's better to make your members private if you can, and only use protected when derived classes are planned and the cost to build and maintain an interface to those private members is too high.

Best practice

Favor private members over protected members.

Public Inheritance

Access specifier in base class	Access specifier when inherited publicly
Public	Public
Protected	Protected
Private	Inaccessible

- There are other types of inheritance – private and protected.
- Focus of this course is **public inheritance**.
- When you inherit a base class publicly,
 - Inherited public members stay public.
 - Inherited protected members stay protected.
 - Inherited private members stay inaccessible.

Self study

```
1 class Base
2 {
3 public:
4     int m_public {};
5 protected:
6     int m_protected {};
7 private:
8     int m_private {};
9 };
10
11 class Pub: public Base // note: public inheritance
12 {
13     // Public inheritance means:
14     // Public inherited members stay public (so m_public is treated as public)
15     // Protected inherited members stay protected (so m_protected is treated as protected)
16     // Private inherited members stay inaccessible (so m_private is inaccessible)
17 public:
18     Pub()
19     {
20         m_public = 1; // okay: m_public was inherited as public
21         m_protected = 2; // okay: m_protected was inherited as protected
22         m_private = 3; // not okay: m_private is inaccessible from derived class
23     }
24 };
25
26 int main()
27 {
28     // Outside access uses the access specifiers of the class being accessed.
29     Base base;
30     base.m_public = 1; // okay: m_public is public in Base
31     base.m_protected = 2; // not okay: m_protected is protected in Base
32     base.m_private = 3; // not okay: m_private is private in Base
33
34     Pub pub;
35     pub.m_public = 1; // okay: m_public is public in Pub
36     pub.m_protected = 2; // not okay: m_protected is protected in Pub
37     pub.m_private = 3; // not okay: m_private is inaccessible in Pub
38
39     return 0;
40 }
```